

ProjWT — WHOOP 4.0 Reverse-Engineering Build Log

Project: ProjWT (personal portfolio project — private) **Owner:** Hamza Khan **Status:** Phase 1–2 (BLE access + protocol) — in progress **Last updated:** 2026-10-01

PRIVATE / PORTFOLIO. This document and the whole ProjWT repo are a personal portfolio piece and are not to be shared publicly.

This is the **living record** of the project — every step, command, result, and decision. It is the source of truth; the PDF/Word versions are generated from this file (see **Regenerating the PDF/Word** at the end). Keep it updated as we go.

1. What we're building (and the safety line)

A personal system that talks directly to a WHOOP 4.0 band over Bluetooth Low Energy (BLE), reads its sensors, and eventually drives:

- **Biometric dashboard** — live heart rate, HRV, SpO₂, skin temp, motion.
- **Gesture control** — recognise wrist gestures from the band's accelerometer.
- **Vehicle control (Tesla Fleet API)** — *supervised only.*
- **Home / IoT automation.**

Target stack (eventual): C++ / Unreal Engine 5. **Phase-1 tooling (now):** Python — fastest way to reverse-engineer and validate the protocol before committing it to C++.

Safety constraint (non-negotiable)

Vehicle control is **active-hold, line-of-sight only**:

- You hold a gesture **while watching the car**; the car moves only while the gesture is actively held.
- Releasing the gesture (a dead-man's-switch) slows or stops the car.
- **No** "no-line-of-sight summon", **no** GPS spoofing, **no** safety-bypass.

That idea was considered and **dropped**.

2. The hardware target

- **Device:** WHOOP 4.0 band (owned by the project author).
- **Advertised name:** WHOOP_4C1656042
- **BLE address (Windows, this session):** D1:86:73:D4:62:86
- ■■ On Windows this address can be a rotating/resolvable random address, so **re-scan each session** rather than hard-coding it (see §8).

- **Battery at bring-up:** 100% (band healthy, charged via the pack; tap → LED).

3. Environment setup (Windows)

The band talks to the PC directly over BLE — **no WHOOP app, no pairing needed**. We drive it from Python.

- 1 **Install Python** (3.10+). During install tick **"Add Python to PATH."**

- 2 **Install the BLE library:**

```
python -m pip install bleak
```

- 1 **Open a terminal in the code folder.** In File Explorer, open the projwt\src folder, click the address bar, type powershell, press Enter.

- 1 Run scripts with python first. PowerShell will ***not*** run ble_scanner_PWT.py on its own (that just opens Notepad). Always:

```
python ble_scanner_PWT.py --name whoop
```

Common beginner errors we already hit and fixed - **"The term 'ble_scanner_PWT.py' is not recognized"** → you forgot the python prefix. Start the command with python. - **"Pressing Enter opens Notepad"** → same cause; Windows is "opening" the file instead of running it. Use python <file>. - **"ModuleNotFoundError: No module named 'bleak'"** → run python -m pip install bleak.

4. The tools we built (in src/, all _PWT)

Script	Purpose
ble_scanner_PWT.py	Discover BLE devices, print name / address / RSSI. Find the band.
gatt_explorer_PWT.py	Enumerate the band's GATT services / characteristics / descriptors.
ble_logger_PWT.py	Subscribe to notify channels and log raw packets to logs/ (JSONL). --label, --seconds.
whoop_protocol_PWT.py	The verified protocol core — framing, CRCs, command IDs, UUIDs.
ble_command_PWT.py	Send one framed command and listen for the reply.
ble_stream_PWT.py	HELLO → START → watch data channel for a continuous stream.
ble_sweep_PWT.py	Sweep a range of command IDs to find which one wakes the data channel.

Script	Purpose
<code>ble_pair_PWT.py</code>	Pair/bond with the band so it will accept our commands (encrypted link).
<code>ble_stop_PWT.py</code>	Send STOP (0x04) to switch the realtime stream + green HR LED back off.
<code>ble_startseq_PWT.py</code>	Try several start sequences in one connection to find the stream trigger.
<code>ble_accel_PWT.py</code>	Find/stream the accelerometer (payload sweep, opcode sweep, live decode).
<code>ble_events_PWT.py</code>	Capture & decode live EVT (04) events, tagged by gesture label.
<code>gesture_PWT.py</code>	Live gesture recognition — tunable FLICK trigger from 0x0e impulses (+ strength filter).
<code>safety_state_machine_PWT.py</code>	L3 fail-closed safety core — pure logic, 7 unit tests, no hardware.
<code>decode_realtime_PWT.py</code>	Decode a saved capture into heart rate + RR intervals (and CSV).

Everything is committed to the private repo `Hamza-Khan-a/lgonquin/projw`, branch `claude/peaceful-dirac-ko6o1i`.

5. What we did, step by step (chronological)

- 1 **Charged and woke the band.** Tapping it showed the LED; battery reached 100%. Windows Bluetooth couldn't see it at first (it wasn't advertising / the WHOOP app's own pairing kept crashing). We bypassed all of that and scanned directly from Python.
- 1 Scanned with `ble_scanner_PWT.py`. Found WHOOP 4C1656042 at D1:86:73:D4:62:86, RSSI $\approx$ -61 dBm (strong, close).
- 1 Mapped the GATT table with `gatt_explorer_PWT.py`. Got the custom WHOOP service and its characteristics (see §6).
- 1 Logged raw notifications with `ble_logger_PWT.py`. **Got 0 packets** at rest — **expected**: the band stays silent until told to stream. Also fixed the ugly Ctrl+C traceback and added `--seconds` auto-stop.
- 1 **Reverse-engineered the packet framing.** Community write-ups disagreed with each other (they were paraphrased summaries), so we **brute-forced the CRC parameters against a real captured packet inside a sandbox** until the framing reproduced byte-for-byte. This is now the verified core (§6).
- 1 Sent our first real command with `ble_command_PWT.py`. **The band** replied

with a valid, CRC-correct packet — **proof our framing is accepted by real hardware (two-way comms confirmed)**. The reply came on the event channel (61080003), 32 bytes, a protobuf-style status/report — **not** a continuous sensor stream.

1 Diagnosed why there was no stream. Reference implementations perform a

HELLO handshake (cmd 0x05) first, then **START**; the continuous data arrives as ~96-byte packets on the data channel (61080004). We had skipped the hello. Built `ble_stream_PWT.py` to do the full sequence.

1 Ran the full HELLO → START sequence (`ble_stream_PWT.py`). The band came

alive — **15 packets in 30s** — but **all on the EVENT channel (61080003)**, a repeating ~2s **status beacon**, **not** the 96-byte sensor stream on the DATA channel (61080004, which stayed silent). Two outcomes:

- **Decoded the beacon's identity fields** (see §6.5): codename `boylston`, firmware **17.2.2.0**, hardware **harvard_r10**.

- **Found & fixed a display bug**: the tools labelled channels by the **last** 8

UUID chars, but every WHOOP characteristic ends in `...b0f8dcc6`, so all channels printed the same label. They differ in the **first** block (6108000x); labelling now uses a clear name map (`EVENT(03)`, `DATA(04)`...).

1 Concluded cmd 0x03 is not the real "start streaming" opcode (the command

IDs were always community guesses). Built `ble_sweep_PWT.py` to brute-force the correct opcode — the same empirical approach that cracked the CRC framing.

1 Swept command IDs (`ble_sweep_PWT.py`) — and learned two big things:

- **The band ignores unauthenticated commands**. Write-**with**-response returned

GATT Protocol Error: Insufficient Authentication (ATT error 0x05 — the command characteristic needs an **encrypted / paired link**). Write-**without**-response (our default) doesn't error, but the band **silently ignores it**. So **none of our commands had ever actually reached the band** — every "reply" so far was the subscribe-triggered status beacon or physical motion.

- **DATA(04) is currently a motion/tap event feed**. The DATA-channel packets

lined up with **tapping / shaking / putting the charger on**, not with any command. They use our verified 0xAA framing, type 0x30, **with an incrementing sequence and a monotonic timestamp** — i.e. timestamped accelerometer/activity events (useful later for gesture detection).

1 Built `ble_pair_PWT.py` and added `--pair` to the sweep/stream tools.

Pairing establishes the encrypted link so the band will honour commands.

1 **PAIRED** — and the band now answers our commands. After pairing, a sweep

with write-with-response showed no more Insufficient Authentication, **and commands** 0x01–0x04 each returned a genuine 32-byte type-0x30 reply on DATA(04) (confirmed: band untouched, timestamps incrementing `0x01e28a92` → ...95 → ...98 → ...9c). cmd 0x0b was a physical tap. **Two-way authenticated command/response is working.**

1 Found the real cause — we were watching the **WRONG** channel. Comparing the

reference (christianmeurer/whoop-reader) char constants against our own scan showed an **off-by-one**: the reference assumes the write char is ...0001, but on our band ...0001 is the **service** and the write char is ...0002, so every characteristic is shifted +1. Aligning by role (write + 4 notify, in handle order) means the real-time sensor stream is on 61080005 — **the channel we had dismissed as "diagnostics". What we called "DATA(04)" is actually the *event* channel (hence one-off command acks + taps). See §6.1 for the corrected map. Also: the reference's start sequence is identical to ours (subscribe → HELLO 0x05 → START 0x03, no payload, response=True), so the opcode was never wrong — the channel was. And every test so far was off-wrist (WHOOP gates the PPG/HR stream on skin contact).**

- 1 Fixed the channel map (DATA = 61080005), solved the Windows idle-drop with a harmless Battery-Level-read keepalive, and built `ble_startseq_PWT.py` to try multiple start sequences in one stable connection.

- 1 ■ **REAL-TIME STREAM UNLOCKED.** The start command is 0x03 with payload 0x01 (bare 0x03 does nothing on fw 17.2.2.0). Data then streams on DATA(05) as ~1 Hz type-0x28 packets **carrying** heart rate + RR intervals. Decoded and verified: one packet reported HR 76 with a single RR of 789 ms, and $60000/789 = 76.0$ exactly. Captured HR climbing 67→78 bpm with RR ~740–820 ms. Built `decode_realtime_PWT.py` to decode captures to HR/RR/CSV.

- 1 HR validated against the owner's real resting BPM — decode confirmed.

- 2 **ACCELEROMETER FORMAT FOUND.** Probing opcodes 0x10–0x1f (`ble_accel_PWT.py --cmd-sweep`) unlocked a high-rate burst of ****96-byte**** type 0x2f ****packets**** on DATA(05). Decoded: ****accelerometer X/Y/Z as three float32 in g units**** at payload offset 36 (a second copy at 52), e.g. a rest vector (-0.125, 0.366, 0.934) → magnitude ****1.01 g**** (gravity). The same window also emitted firmware ****debug-log text**** (type 0x32) and the dump was labelled "Historical Dump" — so 0x16 downloads STORED data. Next: confirm the same 0x2f float format arrives LIVE while moving the wrist.

- 1 **Op notes:** Windows BLE needs a Bluetooth off/on toggle to recover from the "operation was canceled" wedge, and the band must be tapped awake right before running. Fixed a tool crash (formatting unknown packet types) and added live accel/HR decoding + `--extra-cmd`.

- 1 Accel from 0x16 is HISTORICAL, not live. Those 0x2f packets carry a real calendar timestamp (decoded: 2024-09-18), whereas live packets use device-uptime ticks. Re-running 0x16 moving the wrist produced no new accel — 0x16 downloads stored flash records. **Conclusion:** on fw 17.2.2.0, raw accelerometer is recorded to flash and retrieved as history; it does not appear to stream live over BLE via opcodes 0x01–0x1f. Confirmed LIVE signals: HR+RR (type 0x28, 1 Hz) and discrete events on EVT(04) (type 0x30).

- 1 Swept 0x21–0x2f (skipping 0x20 firmware) — no live accel. Every opcode returned only the baseline HR stream (type 0x28); no 0x2f, no new subtype, no rate jump. CONCLUSION (firmware 17.2.2.0): raw accelerometer is NOT exposed as a live BLE stream anywhere in opcodes 0x01–0x2f. It exists only as historical flash records (via 0x16). Live BLE signals available: HR+RR (0x28, 1 Hz) and discrete events on EVT(04) (0x30).

- 1 **Design implication:** a smooth continuous "active-hold" wrist gesture isn't

possible from the band alone over BLE (no live orientation stream). Discrete gestures (tap / double-tap / motion triggers) via EVT (04) are, plus live HR.

1 Live events characterized (labeled captures). Ran `ble_events_PWT.py` for

still / tap / doubletap / flick / raise / armswing. Result:

- 0x0e = **impulse event** (sharp motion): still=0, tap=5, doubletap=6, flick=9, raise=0, armswing=0. It fires on taps/flicks and **ignores slow or large motion** — slow raise and vigorous armswing produced *zero* 0x0e (only HR rose). Green LED = tap registered; taps need to be sharp, flicks are the most reliable primitive.
- 0x21 = once-per-session status/startup event (not a gesture).
- 0x03 / 0x3f = occasional richer "activity report" events (multi-field).
- Confirms: the only live motion signal is **impulse detection**, not continuous orientation.

1 Built the live gesture recogniser (`gesture_PWT.py`): clusters 0x0e

impulses in time → TAP (1) / DOUBLE_TAP (2) / FLICK (≥3), with on-screen output and a stable session. This is the L2 discrete-gesture engine.

1 Impulse strength field confirmed + firmware floor is real. The 0x0e body's

first int16 is an **impact-strength** value: a real flick's initial impulse reads ~7000–31000, while the ringing/settle that follows reads ~1400–4800 (clear gap). So each hard flick can be reduced to one clean FLICK (strength filter + burst grouping + cooldown; defaults: `--strength 6000 --cooldown 0.6`).

But the "must flick hard" requirement is a FIRMWARE floor — the band only emits the event above its own threshold; no host setting lowers it (confirmed; no sensitivity command exists). A host strength filter only improves *accuracy*, not *sensitivity*.

1 Design note: a deliberate firm flick is acceptable — even desirable — as a

discrete **arm / confirm** trigger (you don't want an accidental light motion firing a vehicle/IoT command). Continuous "active-hold" still needs the 2nd sensor per the architecture; discrete confirm = hard flick from the band is fine.

1 Built the L3 safety state machine (`safety_state_machine_PWT.py`) — pure,

hardware-free, fail-closed logic (IDLE/ARMED/HOLDING/STOPPING) consuming gesture (FLICK=arm/disarm), hold (continuous source), heartbeat, tick, and emitting guarded ARM/DISARM/MOVE/STOP commands with a TTL on MOVE. **7 unit tests pass** covering: happy path, continuous-hold MOVE cadence, hold-without-arm (nothing), disarm-stops, arm-timeout, heartbeat-loss stop, and STOPPING can't jump back to HOLDING. Runs with `python safety_state_machine_PWT.py`.

1 ← You are here. Wire the live FLICK recogniser into the state machine as the

arm/confirm input; stand up the continuous-hold source (phone IMU) for the hold input; then a harmless IoT actuator to prove the pipeline end-to-end.

6. The verified protocol (the important part)

6.1 GATT map (from our own scan — authoritative, roles corrected)

Our band has the **service** at ...0001, so the characteristics are shifted +1 versus community write-ups that assume the **write** char is ...0001. Mapping by role (write + 4 notify, in handle order):

UUID (first block)	Role	Notes
61080001	Service	custom GATT service (not a characteristic)
61080002	Command (write)	where we send framed commands
61080003	Response / status	command responses + the ~2s status beacon
61080004	Events	async events — command acks, tap/motion (type 0x30)
61080005	DATA — real-time stream	the 96-byte sensor packets live here
61080007	Diagnostics	
00002a37	Standard Heart Rate	standard BLE HRM characteristic

We originally mis-labelled 61080004 as "data" and 61080005 as "diagnostics" (by matching the reference's numbers instead of its roles). The +1 shift means the real sensor stream is on 61080005. Corrected in `whoop_protocol_PWT.py`.

6.2 Packet framing — empirically verified

```
0xAA | length(2, little-endian) | CRC8(length) | PAYLOAD | CRC32(PAYLOAD, LE)
```

```
length = len(PAYLOAD) + 4          (payload plus the trailing CRC32)
CRC8    = poly 0x07, init 0x00, no reflection, over the 2 length bytes
CRC32   = standard zlib.crc32 over PAYLOAD, stored little-endian
PAYLOAD = [type][seq][cmd][data...]
type    = 0x23 for commands
```

Proof: `build_packet()` reproduces the known-good reference packet exactly:

```
aa100057230423aa8ed469a96d0000005130fef3 ← reference
aa100057230423aa8ed469a96d0000005130fef3 ← our build_packet() ✓ MATCH
```

Run the self-test any time: `python whoop_protocol_PWT.py`.

■■ The MIT reference repo uses a **different, simpler** framing (no CRC8). We do **not** use it — **our** framing is the one the real band actually replied to.

6.3 Command IDs (community-sourced; being verified live)

Cmd	Meaning
0x01	Get battery
0x02	Get device info (fw / serial / hw)

Cmd	Meaning
0x03 + payload 0x01	Start real-time streaming (bare 0x03 does nothing on fw 17.2.2.0)
0x04	Stop real-time streaming
0x05	HELLO / handshake

6.4 Real-time data packet — **VERIFIED (type 0x28, ~1 Hz, on 61080005)**

After START = 0x03 01, the band streams one ~28-byte frame per second on the DATA channel. Our 0xAA framing applies; the **payload** (after unframing) is:

Payload bytes	Field	Notes
[0]	packet type	0x28
[1]	subtype	0x02 = realtime metrics
[2:6]	timestamp	uint32 LE, device uptime seconds
[6:8]	aux	uint16 LE — not yet decoded (PPG/activity?)
[8]	heart rate	bpm (uint8)
[9]	N	number of RR intervals that follow
[10 : 10+2N]	RR intervals	N × uint16 LE, milliseconds (beat-to-beat → HRV)

Consistency check: a packet with HR=76 carried a single RR of 789 ms, and $60000 \div 789 = 76.0$. A capture showed HR climbing 67→78 bpm with RR ~740–820 ms. Decoder: `python decode_realtime_PWT.py <logfile>`.

This replaces the earlier *tentative* 96-byte layout from the reference — our firmware streams these compact 28-byte HR/RR packets instead.

6.8 Accelerometer packet — **type 0x2f (96 bytes, on 61080005)**

Unlocked by probing opcodes 0x10–0x1f. The accelerometer is reported as **float32 (little-endian), in g units**:

Payload bytes	Field
[0]	packet type (0x2f)
[1]	subtype (0x0c)
[36:40] [40:44] [44:48]	accel X / Y / Z — float32, g units
[52:64]	second X/Y/Z copy (filtered vs raw)

Verified: a rest packet decoded to $(-0.125, 0.366, 0.934)$, magnitude **1.01 g** (gravity). Parser: `whoop_protocol_PWT.parse_accel()`; live view: `python ble_accel_PWT.py <ADDR> --start-data 01 --extra-cmd 0x16 --label tilt`.

type 0x32 packets seen alongside are firmware **debug-log text** (ASCII), not sensor data. **Important:** the 0x16 dump is HISTORICAL — these 0x2f packets carry a real calendar timestamp (2024-09-18), so the accel values are stored flash records, not a live feed. Live raw accel has not been found over BLE on this firmware (opcodes 0x01–0x1f); see §9 for the gesture-control fork.

6.9 Live events — type 0x30 on EVENT_CHAR (61080004)

Discrete events. Layout: [0]=type 0x30, [1]=seq, [2]=**report id**, [3]=0, [4:8]=uptime ts, [8:]=body. Report ids observed, from labeled captures:

Report id	Meaning	Fires on
0x0e	Impulse (tap / flick) — the gesture primitive	sharp wrist motion only
0x21	Session status / startup	once on connect
0x03, 0x3f	Activity report (multi-field body)	occasionally during motion

Key behaviour: **slow or large motion produces no event** (a slow arm-raise and a vigorous arm-swing both yielded zero 0x0e — only heart rate rose). So the band's only live motion signal is impulse detection. Gesture vocabulary is built by clustering 0x0e in time (`gesture_PWT.py`): 1 → TAP, 2 → DOUBLE_TAP, ≥3 → FLICK.

Captured **event** packet (32 bytes on 61080003, protobuf body) for reference:

```
aa 1c 00 ab 30 06 1d 00 2d 48 e1 01 20 1a 0c 00 32 01 00 00 02 00 42 02 0b 00 01 00 8c 60 8f ce
```

6.5 Status beacon (EVENT channel 61080003) — decoded identity

When we subscribe, the band emits a ~97-byte **status beacon every ~2 seconds** on the EVENT channel. It is a protobuf body (starts 08 02 a6 02 ...) carrying the device identity plus a changing counter near the tail. Decoded ASCII fields:

Field	Value
Codename	boylston
Firmware / version	17.2.2.0
Hardware revision	harvard_r10

This is a **heartbeat/status** feed, not the per-sample sensor stream. The per-sample sensor data (PPG / accel / HR) is expected as ~96-byte packets on the DATA channel 61080004, which is still silent — see §9 next steps.

6.6 Authentication requirement (the key blocker)

The command characteristic (61080002) requires an **encrypted/paired BLE link**. Evidence: write-with-response returns GATT Protocol Error: Insufficient Authentication (ATT error 0x05). Write-without-response doesn't raise, but the band silently discards the write. **Therefore commands only take effect after we pair/bond** (`ble_pair_PWT.py`, or `--pair`). Until then the band

only emits the status beacon and motion events, regardless of what we send.

6.7 DATA(04) event packets (motion-triggered) — framing confirmed

The DATA-channel packets observed so far are our 0xAA frame with type 0x30:

```
aa | len(2 LE) | crc8(len) | 30 | seq | .. | timestamp(uint32 LE) | .. | crc32
```

Example: aa 10 00 57 30 24 0e 00 c8 87 e2 01 80 44 00 00 ... → len=16, crc8 OK, type=0x30, seq=0x24, timestamp=0x01e287c8 (monotonic). These fire on tap/shake/charger — accelerometer/activity events, not the continuous PPG stream we still need to unlock (via pairing + the right opcode).

7. Legal / attribution — do the referenced authors need to know?

Short answer: no obligation to notify anyone, and you don't have to publish that you used their work.

- The reference we consulted, christianmeurer/whoop-reader, is MIT-licensed.

The MIT license lets you use, copy, modify, and even redistribute the code — including in a closed/private project — **with one condition**: *if you redistribute their source or substantial portions of it, you must keep their copyright notice + the MIT license text with it.*

- **You are not "redistributing" it.** We only **read** it for reference and wrote

our **own** implementation (in fact with *different, independently verified* framing). So the MIT condition isn't even triggered.

- **The author is never automatically notified.** GitHub doesn't tell someone when you read, clone, or reference their repo. There's no tracking, no callback.

- **Do you *have* to "let it be known"?** No. But crediting them is good

practice and costs nothing — so we've added a short credit note in whoop_protocol_PWT.py and here. It also makes the portfolio piece look more professional (shows you research prior art and respect licenses).

- The other names in the space (jogolden/whoomp, bwanShiTong/reverse-engineering-whoop) — same logic. jogolden/whoomp couldn't be added to this session (not found/accessible), so we used the MIT whoop-reader instead.

Bottom line: keep the credit note (already added), don't paste their source verbatim into your repo, and you're completely fine — legally and ethically.

8. Reconnecting after a break (start-of-session checklist)

The band is stateless from our side — nothing persists on the PC. Next time you sit down:

- 1 **Charge the band** if it's been idle a while; tap it to confirm the LED.
- 2 **Open PowerShell in the code folder** — File Explorer → projwt\src folder
→ type powershell in the address bar → Enter.

1 **Re-scan for the band** (the address may have changed):

```
python ble_scanner_PWT.py --name whoop
```

Copy the address it prints (e.g. D1:86:73:D4:62:86).

1 **(Optional) confirm the GATT map is unchanged:**

```
python gatt_explorer_PWT.py <ADDRESS>
```

1 **Pair once** (only needed the first time, or after --unpair; the bond persists across sessions):

```
python ble_pair_PWT.py <ADDRESS>
```

1 **Find the stream opcode** (current task — see §9), band held still:

```
python ble_sweep_PWT.py <ADDRESS> --pair --with-response
```

Once we know the right opcode, stream with:

```
python ble_stream_PWT.py <ADDRESS> --pair --with-response --seconds 30
```

1 Send Claude the newest file in logs\ so we can decode/continue.

If bleak is missing again (new machine): `python -m pip install bleak`. If nothing is found in the scan: make sure the band is charged and near the PC, and that no other app (the WHOOP phone app) is holding the connection.

9. Current status & next steps

Done

- Physical bring-up, BLE discovery, full GATT map.
- Packet framing reverse-engineered **and verified both ways** (we sent a command and the real band returned a valid CRC-checked reply).
- Confirmed two-way comms live; decoded the device-identity status beacon (firmware 17.2.2.0, hardware harvard_r10).
- Tooling for scan / explore / log / command / stream / sweep.

Done (new)

- **Pairing works** (Windows ConfirmOnly ceremony); authenticated link accepts commands.
- **Windows idle-drop solved** with a Battery-Level-read keepalive.
- **Real-time HR + RR streaming works** — START = 0x03 01, data on DATA(05) as ~1 Hz type-0x28 packets; decoded and verified against the owner's resting BPM.
- **Accelerometer format decoded** — type-0x2f packets, X/Y/Z float32 (g).

Finding: live raw accel is NOT available over BLE (swept 0x01–0x2f). Accel is historical-only. So gesture control uses the live EVT(04) events + HR.

Done (new): live events characterized (0x0e = tap/flick impulse) and a live gesture recogniser built (gesture_PWT.py: TAP / DOUBLE_TAP / FLICK).

Next

1 Tune the recogniser on-wrist (tap the band awake first):

```
python gesture_PWT.py <ADDRESS> --seconds 120
```

Flick a few times, tap sharply, double-tap. Adjust --gap (cluster window) and --flick (impulses for a flick) until TAP vs FLICK feel right. Send Claude a couple of logs\events_*.jsonl so we can decode the 0x0e body (impact field).

1 Build the **L3 safety state machine** (unit-tested, no hardware) driven by the recogniser: e.g. FLICK = arm/disarm, with fail-closed guards.

1 Decode the HR aux field; decode historical accel (0x2f) offline.

2 Port the verified protocol to C++ for the UE5 phase.

Windows gotchas (keep handy)

- "operation was canceled by the user" / repeated drops → toggle Bluetooth OFF/ON.
- Tap the band awake immediately before running any script.
- **Green HR LED stays on after a run?** START (0x03 01) powers the optical HR

sensor (green LED); tools send STOP (0x04) on a clean finish, but a Ctrl+C / closed window skips it, so the band keeps streaming (battery drain, LED on even off-wrist). Fix: python ble_stop_PWT.py <ADDRESS> (or just let it idle/ disconnect and it stops on its own). Behaviour of the band, driven by our last command — not a fault.

10. Regenerating the PDF / Word versions

This Markdown file is the master. The PDF and Word copies are generated from it with a small pure-Python script (no LibreOffice needed, so it runs on your Windows machine too). One-time setup:

```
python -m pip install reportlab python-docx
```

Then, from the repo root, regenerate BUILD_LOG_PWT.pdf and BUILD_LOG_PWT.docx any time you edit this file:

```
python Tools/export_docs_PWT.py
```

Re-run it whenever you update this log so the shareable copies stay in sync.

ProjWT build log — private portfolio project. Keep this file updated.